

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design
Weightage: 20%

QUESTIONS:

Total Marks: 25

1.A company needs to develop a web service for a mobile and web application. Analyze both **REST** and **SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

1. REST vs SOAP – Selection of an Appropriate Web Service

Introduction

A company needs to develop a web service that will be accessed by both mobile and web applications. The two common approaches are **REST** and **SOAP**. Both can provide communication between clients and servers, but they differ in performance, scalability, flexibility, and security.

Feature	REST	SOAP
Architecture	Architectural style	Protocol
Data format	Mainly JSON, also XML	XML
Performance	Faster and lightweight	Comparatively slower

Feature	REST	SOAP
Scalability	Highly scalable	Less flexible for large-scale web applications
Security	HTTPS, OAuth, JWT	WS-Security, HTTPS
Complexity	Simple	More complex
Mobile applications	Very suitable	Less suitable
Browser/Web applications	Highly suitable	Suitable
Bandwidth	Lower	Higher
Development	Easy	More complex

Selected Approach: REST

For a mobile and web application, **REST API** is the most appropriate choice.

REST Architecture

Mobile Application

|

|

Web Application

|

↓

REST API

|

↓

Application Server

|

↓

Database

Example REST Request

GET /api/users/101

Response:

```
{  
  "id": 101,  
  "name": "Rahul",  
  "email": "rahul@example.com"  
}
```

Why REST is Selected

1.

Performance:

REST commonly uses JSON, which is lightweight and requires less bandwidth. This provides faster communication, especially for mobile applications.

2.

Scalability:

REST services are generally stateless. Each request contains the information required to process it, making it easier to scale servers horizontally.

3.

Flexibility:

REST can be consumed by different clients such as Android, iOS, web browsers, and desktop applications.

4.

Security:

HTTPS can encrypt communication. Authentication can be implemented using OAuth 2.0 or JWT-based mechanisms.

5.

Easy

Development:

REST APIs are simple to develop, test, and integrate.

Alternative

SOAP can be selected when the application requires strict contracts, advanced enterprise security, reliable messaging, or transaction support.

Conclusion

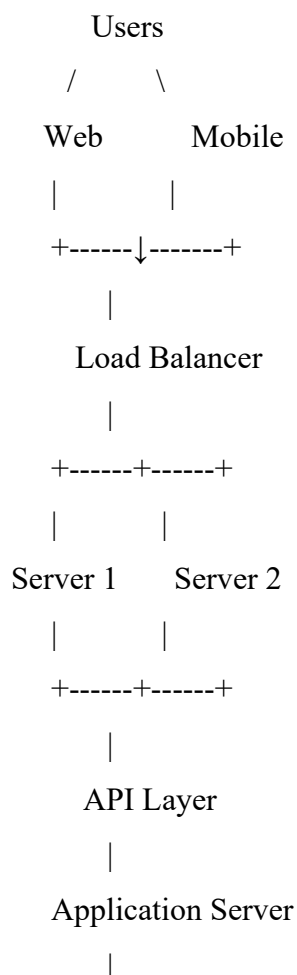
For a modern mobile and web application, **REST is preferred because it provides better performance, scalability, flexibility, and simpler development while still supporting strong security mechanisms.**

2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

Introduction

An online platform with a large number of simultaneous users needs an architecture that can handle high traffic without becoming slow or unavailable. A scalable architecture distributes requests across multiple servers and prevents a single component from becoming a bottleneck.

Basic Architecture



Database

Working

1. Users access the web or mobile application.
2. Requests are sent to the **Load Balancer**.
3. The load balancer distributes requests among multiple servers.
4. The API layer receives and processes the requests.
5. The application servers perform business operations.
6. The database stores and retrieves information.
7. The response is returned to the client.

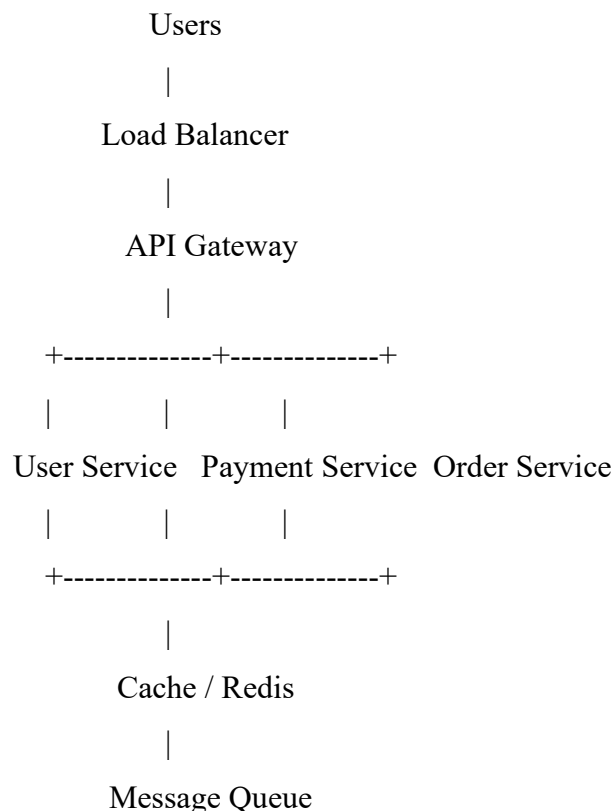
Problems with the Basic Design

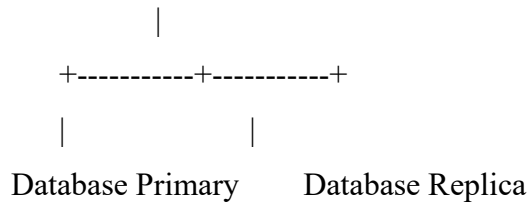
If all requests depend on one database or one application server, the system may experience:

- Slow response time
- Server overload
- Database bottlenecks
- Service failure
- Reduced availability

Alternative Design: Cloud-Based Microservices Architecture

A more scalable design can use **microservices, caching, load balancing, database replication, and message queues**.





Advantages of the Alternative

1.HorizontalScaling:

Additional servers can be added when the number of users increases.

2.LoadBalancing:

Traffic is distributed across multiple servers.

3.Caching:

Frequently accessed data can be stored in a cache, reducing database requests.

4.DatabaseReplication:

Multiple database replicas can handle read requests and improve availability.

5.Microservices:

Individual services can be scaled independently.

6.MessageQueue:

Time-consuming background tasks can be processed asynchronously.

Conclusion

The microservices-based architecture provides better **scalability, availability, fault isolation, and performance** for a platform with a large number of simultaneous users.

3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.

Introduction

A banking application handles highly sensitive information such as account details, passwords, transaction information, and financial data. Therefore, communication between the client and server must provide strong confidentiality, integrity, authentication, and reliability.

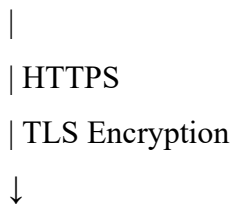
Protocol	Security	Reliability	Performance	Suitability
HTTP	Low	Good	High	Not suitable for banking
HTTPS	High	High	High	Highly suitable
FTP	Low/Medium	Good	Medium	Not suitable
SMTP	Medium	Good	Medium	Not suitable
WebSocket over TLS	High	High	Very high	Useful for real-time communication

Selected Protocol: HTTPS

HTTPS (HTTP Secure) is the most suitable protocol for normal banking client-server communication.

Architecture

Banking Client



Web/API Server



Authentication Server



Banking Database

Why HTTPS is Selected

1. Confidentiality:

TLS encrypts data transmitted between the client and server. Attackers cannot easily read intercepted communication.

2. Data Integrity:

TLS helps detect modification of data while it is being transmitted.

3. Authentication:

Digital certificates help the client verify that it is communicating with the intended server.

4. Reliability:

HTTPS operates over TCP, which provides reliable delivery of data.

5.Performance:

Modern TLS and HTTP versions provide efficient encrypted communication. Persistent connections and HTTP/2 or HTTP/3 can further improve performance.

Additional Security Measures

A banking application should also use:

- Multi-factor authentication
- Strong password policies
- Secure session management
- Authorization checks
- Rate limiting
- Input validation
- Encryption of sensitive data at rest
- Transaction monitoring
- Secure logging

Alternative

For real-time banking notifications, **WebSocket over TLS** can be used when persistent two-way communication is required.

Conclusion

HTTPS with modern TLS is the most suitable protocol for banking applications because it provides confidentiality, integrity, authentication, reliability, and good performance.

4.An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.

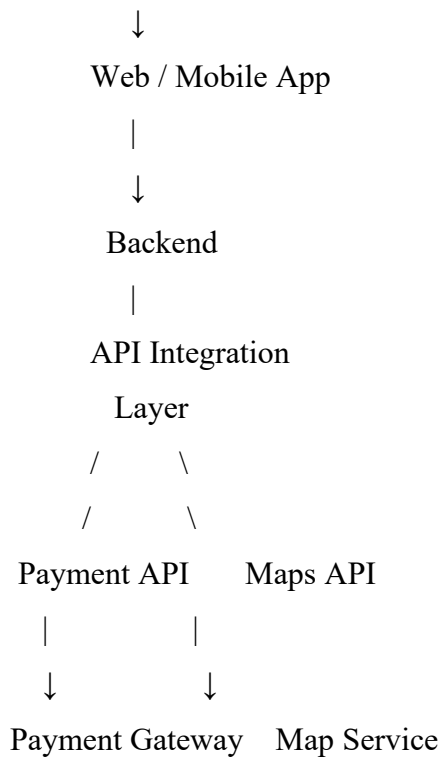
Introduction

Modern applications often depend on third-party services such as payment gateways, map services, SMS services, email services, and authentication providers. A well-designed API integration strategy should securely handle requests, authentication, responses, failures, and retries.

API Integration Architecture

User

|



Request Handling

The following steps can be used:

1. The user performs an action.
2. The application sends the request to the backend.
3. The backend validates the request.
4. The integration layer creates the third-party API request.
5. Authentication credentials are added.
6. The request is sent to the external service.
7. The response is received.
8. The response is validated and processed.
9. A suitable result is returned to the application.

Example Payment Request

Example Payment Request

POST /payment

Content-Type: application/json

Authorization: Bearer ACCESS_TOKEN

```
{
  "amount": 5000,
  "currency": "INR",
  "orderId": "ORD1001"
```

```
}
```

Authentication

Different APIs may use:

- API keys
- OAuth 2.0
- JWT
- Access tokens
- Digital certificates

Sensitive credentials should **not be stored directly in frontend code**. They should be securely managed on the server side.

Response Processing

A successful response might contain:

```
{  
  "status": "success",  
  "transactionId": "TXN1001"  
}
```

The backend should:

1. Validate the response.
2. Check the status.
3. Store the transaction result.
4. Return a simplified response to the client.
5. Handle failures appropriately.

Alternative Approach: API Gateway + Queue

For better reliability, an **API Gateway** and **message queue** can be introduced.

Client

|

↓

API Gateway

|

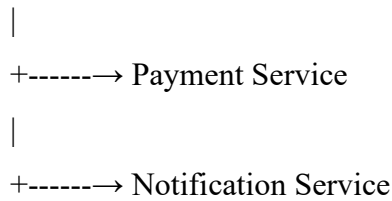
↓

Backend

|

↓

Message Queue



Advantages

- Centralized authentication
- Rate limiting
- Monitoring
- Retry mechanisms
- Better fault isolation
- Asynchronous processing
- Reduced dependency between services

Conclusion

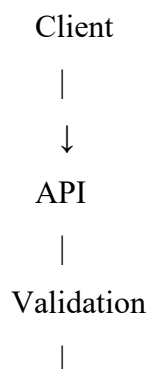
A centralized API integration layer provides secure and controlled communication with third-party services. Adding an API Gateway, caching, queues, and retry mechanisms can further improve **efficiency, reliability, and scalability**.

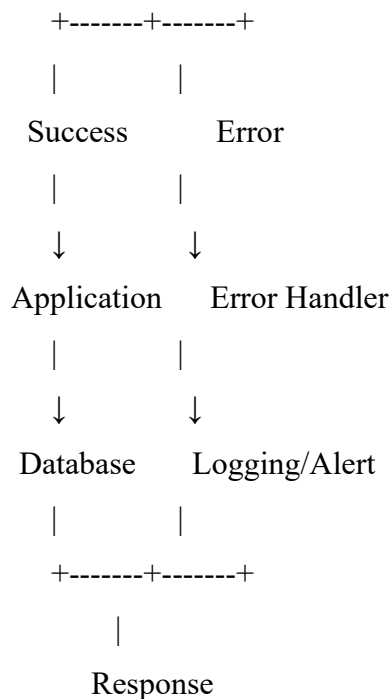
5.A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Introduction

Web services may experience errors because of invalid requests, network failures, server crashes, database problems, or third-party service failures. A proper error-handling and fault-tolerance mechanism helps maintain system availability and provides a smooth user experience.

Error Handling Architecture





Types of Errors

1. Client Errors

Examples:

- Invalid input
- Missing parameters
- Unauthorized request
- Resource not found

Common HTTP status codes:

400 – Bad Request

401 – Unauthorized

403 – Forbidden

404 – Not Found

2. Server Errors

Examples:

- Application failure
- Database failure
- Internal exception

Common status codes:

500 – Internal Server Error

502 – Bad Gateway

503 – Service Unavailable

504 – Gateway Timeout

Error Handling Mechanism

Step 1: Input Validation

Validate all incoming requests before processing them.

Request → Validate → Process

Invalid requests should be rejected with a meaningful error message.

Step 2: Centralized Error Handler

Use a centralized error-handling mechanism to generate consistent responses.

Example:

```
{
  "status": 400,
  "error": "Invalid Request",
  "message": "Patient ID is required"
}
```

Step 3: Logging

Record important information such as:

- Error type
- Time of error
- Request ID
- Service name
- Server information

Logs help developers identify and resolve problems.

Step 4: Retry Mechanism

Temporary failures can be handled using controlled retries.

Request

↓

Failure

↓

Retry

↓

Success → Continue

↓

Failure

↓

Return Error

Retries should generally use **exponential backoff** rather than repeatedly sending requests immediately.

Step 5: Timeout

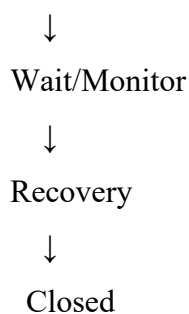
Set a maximum waiting time for external services. If the service does not respond within that time, the request should fail gracefully instead of keeping the user waiting indefinitely.

Alternative Fault-Tolerance Strategies

1. Circuit Breaker

If an external service repeatedly fails, the circuit breaker temporarily stops sending requests to it.

Normal → Failure → Open



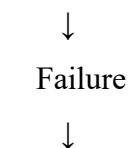
This prevents cascading failures.

2. Fallback

If the primary service is unavailable, provide an alternative response.

Example:

Primary Map API



Backup Map API

3. Load Balancing

Distribute requests among multiple servers to prevent a single server from becoming overloaded.

4. Caching

Frequently accessed information can be cached so that temporary backend failures do not always affect users.

5. Health Checks

Regularly monitor service health and automatically remove unhealthy servers from the traffic pool.

6. Graceful User Experience

Instead of showing technical errors such as:

NullPointerException

the application should display:

"Sorry, we couldn't process your request right now.

Please try again in a few moments."

Conclusion

A combination of **input validation, centralized error handling, logging, retries, timeouts, circuit breakers, fallbacks, load balancing, caching, and health monitoring** provides strong fault tolerance. This minimizes service interruptions and ensures a smoother user experience.

Code of Conduct:

I __P.Harika (192311243)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Harika

RUBRICS FOR CALCULATION:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives

Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect